Estructuras de Datos Dinámicas

Listas Enlazadas Simples y Dobles

Fecha de revisión:

Ing. Sofía Pérez

Agenda

1. Estructuras estáticas vs dinámicas
2. Nodo y estructura autoreferenciada
3. Lista simplemente enlazada
   a. Concepto y representación
   b. Operaciones: inserción, recorrido y eliminación.
4. Lista doblemente enlazada
   a. Ventajas y nueva estructura
   b. Operaciones
5. Lista simple vs lista doble

Estructuras estáticas (arreglo)

Son aquellas colecciones de datos cuyo **tamaño se define en tiempo de compilación y no puede cambiar durante la ejecución.**

```
int a[100]; // Capacidad máxima fija: 100 enteros
```

Limitación Principal:

Tamaño fijo. Si necesitamos 101 elementos, fallamos. Si sólo usamos 10, desperdiciamos memoria.

Estructuras dinámicas

Son aquellas colecciones de datos cuyo **tamaño puede aumentar o disminuir durante la ejecución del programa.**

¿Cómo funcionan?: No usan un bloque único de memoria. En su lugar, **solicitan memoria "a demanda" para cada nuevo elemento y los "enlazan" entre sí.**

Ventaja Principal:

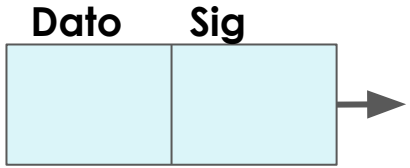
Uso eficiente de la memoria y tamaño variable. Solo usamos lo que necesitamos.

El Nodo

Un **nodo es una estructura** que contiene dos cosas:

- **Datos:** La información útil que queremos almacenar (un número, un nombre, otra estructura, etc.).
- **Un Puntero:** Una referencia o "flecha" que apunta a la ubicación en memoria del siguiente **nodo.**

```
typedef struct Nodo {
    int valor;              // El dato
    struct Nodo* sig;   // Puntero al siguiente Nodo
} Nodo;
```



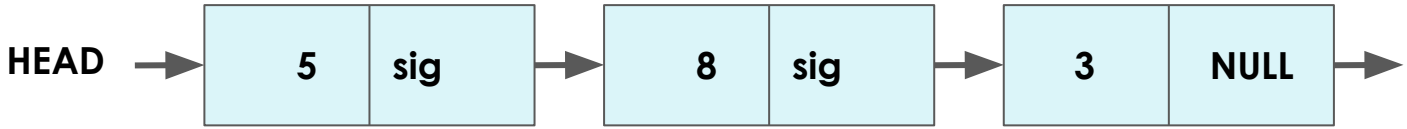
Estructura autoreferenciada

Es una estructura autoreferenciada es aquella **que contiene, entre sus miembros, al menos un puntero a una estructura de su mismo tipo.**

```
typedef struct Nodo {
    int valor;              // 1. Un miembro para los datos.
    struct Nodo* sig;       // 2. Un miembro que es un PUNTERO
                            //    al mismo tipo de estructura.
} Nodo;
```

Lista simplemente enlazada

Es una **colección lineal de nodos** donde cada nodo apunta únicamente al siguiente.



- **Head (o Puntero Principal):** Un puntero que siempre apunta al primer nodo de la lista. Si es NULL, la lista está vacía.
- **Nodos Intermedios:** Cada uno apunta al siguiente.
- **Último Nodo:** Su puntero "siguiente" apunta a NULL, marcando el final de la lista.

Lista simplemente enlazada

Al conectar varios nodos, formamos una **lista simplemente enlazada**.

Es **"simple"** porque la navegación es en un solo sentido.

Siempre necesitamos un punto de partida, ese es el puntero **"head"** Y para saber cuándo detenernos, el **último nodo** apunta a **NULL.**

Si perdemos el **"head"**, perdemos la lista entera.

Creamos una lista de nodos

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Nodo {
        int valor;
        struct Nodo* sig;
}Nodo;

// prototipos de funciones: agregar(), mostrar(), eliminar(), buscar()....

int main() {
        Nodo* p=NULL;   // inicializamos la lista llamada p
        return 0;
}
// desarrollo de funciones
```

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Nodo {
        int valor;
        struct Nodo* sig;
}Nodo;

// prototipos de funciones: agregar(), mostrar(), eliminar(), buscar()....
void agregar(Nodo** p, int v);

int main() {
        Nodo* p=NULL;  // inicializamos la lista llamada p
        return 0;
}
// desarrollo de funciones
void agregar(Nodo** p, int v) {
...
}
```

La función **agregar()** *debe ser capaz de modificar este puntero p (head)*. Específicamente, cuando la lista está vacía (p = NULL), la función debe cambiar p para que apunte al primer nodo.

Puntero a puntero ¿Por qué Nodo** para agregar()?

¿Por qué Nodo p?**

// PROTOTIPO CORRECTO:

```
void agregar(Nodo** direccion_de_p, int v);
```

- Le pasamos la dirección de memoria de p a la función (&p).
- El parámetro direccion_de_p ahora sabe "dónde vive" el puntero p original.
- Para modificar el p original, usamos el operador *: la expresión *direccion_de_p nos da acceso a la variable p de main.

¿Qué pasaría si usáramos Nodo*?

// PROTOTIPO INCORRECTO:

```
void agregar(Nodo* copia_de_p, int v);
```

- La función recibiría una copia de p.
- Cualquier cambio a copia_de_p se perdería al terminar la función.
- La p original en main seguiría siendo NULL.

Función: agregar al final (append)

Esta función permite **añadir un nuevo nodo al final** de la lista.

Hay 2 casos a contemplar:

Caso A: Agregando **el primer nodo** (Lista Vacía)

Caso B: Agregando **un nodo a una lista existente**

Función: agregar al final (append) - CASO A

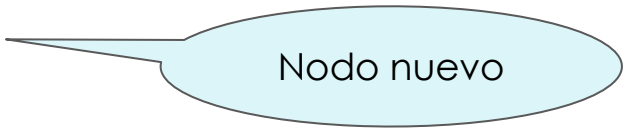
Caso A: Agregando el primer nodo **(agregar(&p, 5))**

Partimos de una lista vacía. **p -> NULL**

1) Creamos el nuevo nodo en memoria. Aún no está conectado a nada.

```
Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo));
```

[5 | NULL]



Función: agregar al final (append) - CASO A

2) Preguntamos si ¿La lista está vacía?. En este caso, ***p es NULL**, así que la respuesta es SÍ. Entramos en el if.

if(*p == NULL)

3) Le decimos al **puntero principal p** que ahora **debe apuntar a nuestro nuevo nodo**. Ahora el nodo nuevo, es el primer nodo.

p → [5 | NULL]

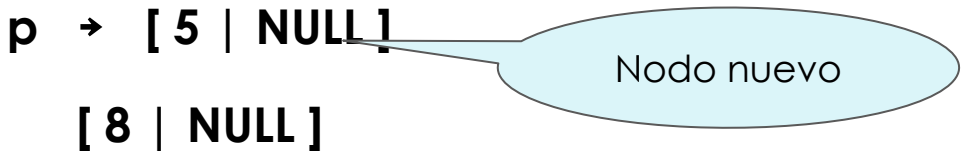
Función: agregar al final (append) - CASO B

Caso B: Agregando un nodo a una lista existente **(agregar(&p, 8))**

Partimos de la lista con el nodo 5.

p → [5 | NULL]

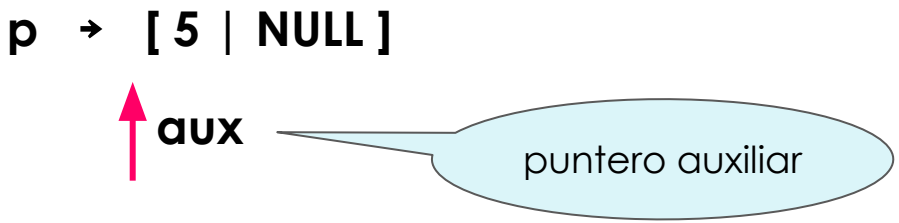
1) Creamos el nuevo nodo 8, que sigue "flotando".



Función: agregar al final (append) - CASO B

2) Preguntamos si ¿La lista está vacía?. ***p** ahora apunta al nodo 5, así que **NO es NULL.** Saltamos al else.

3) Creamos el **puntero auxiliar aux** para recorrer la lista. Lo posicionamos al inicio.



Función: agregar al final (append) - CASO B

4) El código revisa si el nodo aux apunta a otro nodo.

En este caso, **aux->sig es NULL.**

La condición del while es falsa, por lo que el bucle no se ejecuta. aux se queda donde está, apuntando al último nodo (que en este caso, también es el primero).

while(aux->sig != NULL)

Función: agregar al final (append) - CASO B

5) El campo sig del nodo donde está aux (el nodo 5) pasa a apuntar al nuevo nodo (el nodo 8).

p → [5 | sig] → [8 | NULL]

Cualquier llamada posterior a agregar (con 7, 3, 9...) seguirá la lógica del Caso 2, con la única diferencia de que el bucle while sí tendrá que ejecutarse para mover aux hasta el final de la lista antes de poder enganchar el nuevo nodo.

p → [5 | sig] → [8 | sig] → [7 | sig] → [3 | sig] → [9 | NULL]

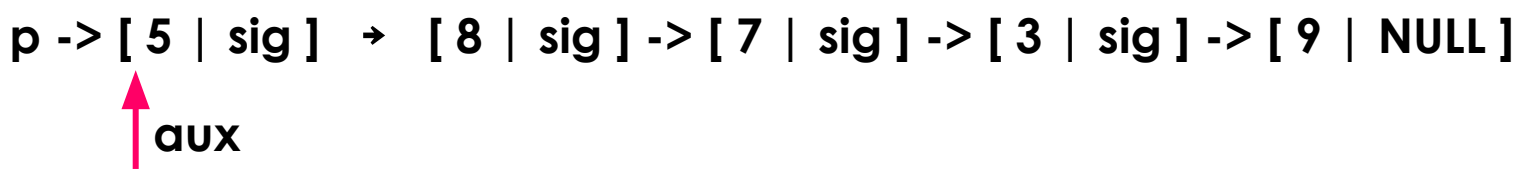
```
//Desarrollo de la función
void agregar(Nodo** p, int v){
        Nodo *nuevo = (Nodo*)malloc(sizeof(Nodo)); // creamos el nuevo nodo y le reservamos memoria
        //inicializamos los valores del nodo
        nuevo->valor=v;
        nuevo->sig=NULL;
        if(*p==NULL){ // si la lista está vacía entonces hacemos que p apunte al nuevo nodo
                *p=nuevo; // el nuevo nodo pasa a ser el primero (head)
        }else{
                //Si no está vacía la lista creamos un nodo auxiliar para no perder el head (es decir el p)
                Nodo* aux=*p;
                while(aux->sig!=NULL){ // recorremos la lista hasta llegar al último nodo
                        aux=aux->sig;     // avanzamos a aux al próximo nodo
                }
                aux->sig=nuevo; // como aux apunta al último entonces su siguiente sera el nuevo nodo
        }
}
```

Función: mostrar

Esta función permite **recorrer la lista** desde el principio hasta el fin e imprimir en pantalla el valor de cada nodo. Llamada: **mostrar(p):**

1) Creamos un puntero auxiliar aux y lo hacemos apuntar al mismo lugar que p, es decir, al inicio de la lista. (nunca movemos p para no perder la referencia de donde comienza la lista)

Nodo* aux = p;



Función: mostrar

2) Comienza el bucle. **while(aux != NULL)**. La condición es: ¿aux apunta a un Nodo o a NULL?. Cómo aux apunta al nodo 5, la condición es **verdadera** y entramos al bucle.

Imprimimos el valor al que apunta aux (un 5 la 1era vez). **printf("%d -> ", aux->valor);**

Movemos aux para que apunte a lo que apuntaba el sig del nodo actual. Es decir, aux avanza al siguiente nodo. **aux = aux->sig;**

p -> [5 | sig] -> [8 | sig] -> [7 | sig] -> [3 | sig] -> [9 | NULL]



aux

Función: mostrar

3) Volvemos a evaluar la condición. aux ahora apunta al nodo 8, así que la condición es verdadera.

Se imprime el valor de aux (un 8) y aux avanza al nodo 7.



4) El proceso se repite. El bucle continuará, imprimiendo 7 -> y luego 3 -> y finalmente 9 ->. Después de imprimir el 9, aux avanzará al sig del último nodo, que es NULL.

Función: mostrar

5) En el último paso, aux quedó apuntando a NULL.

while(aux != NULL)

p -> [5 | ...] -> [9 | NULL] aux -> NULL

Ahora la condición **aux != NULL** es **falsa**. El bucle termina.

Fuera del bucle, imprimimos "NULL" para representar visualmente el final de la lista.

En pantalla: **5 -> 8 -> 7 -> 3 -> 9 -> NULL**

```
// Prototipado
void mostrar(Nodo* p);

//Desarrollo de la función
void mostrar(Nodo* p){
        // creamos un nodo auxiliar para recorrer la lista
        Nodo* aux = p;
        // recorremos la lista mientras aux no sea NULL
        while(aux != NULL){
                // imprimimos el valor del nodo actual
                printf("%d -> ", aux->valor);
                // avanzamos al siguiente nodo
                aux = aux->sig;
        }
        // al terminar mostramos el final de la lista
        printf("NULL\n");
}
```

Aquí usamos Nodo* porque solo leemos la lista, no la modificamos.

Función: eliminar

Esta función permite encontrar y remover un nodo con un valor específico.

Llamada: **eliminar(&p, 8);**

Estado inicial de la Lista

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]

Función: eliminar

1) Creamos dos punteros: **aux** para recorrer la lista (comienza en la cabeza) y **ant** (anterior) que lo sigue un paso atrás (comienza en NULL).

Nodo* aux=*p;

Nodo* ant=NULL;

ant -> NULL

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]



aux

Función: eliminar

2) Buscamos el nodo (Iteración 1 del while). El bucle comprueba si el valor de aux (5) es el que buscamos (8). No lo es. Por lo tanto, avanzamos los punteros: ant se mueve a donde está aux y aux avanza al siguiente.

```
while((aux!=NULL) && (aux->valor!=v)){ // Verdadero, 5 != 8
      ant=aux;
      aux=aux->sig;
}
```



Función: eliminar

3) Encontramos el nodo (Fin del while). El bucle vuelve a comprobar. Ahora el valor de aux (8) es igual al que buscamos (8). La condición del while se vuelve falsa y el bucle termina.

Hemos encontrado el nodo a eliminar.

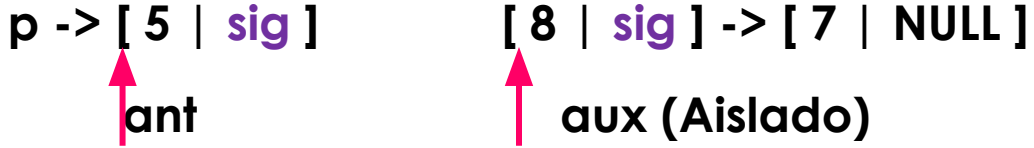
```
while((aux!=NULL) && (aux->valor!=v)){ // Falso, 8 == 8
```

Función: eliminar

4) Eliminamos el nodo y unimos los nodos. Como ant no es NULL (apunta al nodo 5), entramos en el if.

```
if(ant!=NULL){ // Verdadero
    ant->sig=aux->sig;
}
```

Hacemos que el sig del nodo anterior (ant) se salte al nodo actual y apunte directamente al siguiente (aux->sig). El nodo 8 queda desconectado de la lista.



Función: eliminar

4) Liberamos la memoria del nodo 8 que está fuera de la lista, pero sigue ocupando espacio en memoria. La función free() devuelve esa memoria al sistema para que pueda ser reutilizada.

free(aux);

p -> [5 | sig] -> [7 | NULL]

```
void eliminar(Nodo** p, int v){
    // creamos un nodo auxiliar para recorrer y otro para guardar el anterior
    Nodo* aux=*p;
    Nodo* ant=NULL;
    // buscamos el nodo que contiene el valor v
    while((aux!=NULL) && (aux->valor!=v)){
        ant=aux;  // guardamos el nodo anterior
        aux=aux->sig; // avanzamos al siguiente
    }
    if(aux!=NULL){  // si encontramos el nodo (aux no es NULL)
        if(ant!=NULL){ // si no es el primero, enlazamos el anterior con el siguiente
            ant->sig=aux->sig;
        }else{
            *p=aux->sig; // si es el primero, movemos el head al siguiente nodo
        }
        free(aux); // liberamos la memoria del nodo eliminado
    }
}
```

Función: buscar

La función permite recorrer la lista para encontrar el primer nodo que contenga un valor específico (v). Si lo encuentra, devuelve un puntero a ese nodo; si no lo encuentra, devuelve NULL.

Esta función es de "solo lectura", es decir, no modifica la lista de ninguna manera.

Estado Inicial de la Lista:

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]

Función: buscar

Caso A: El valor es encontrado **(buscar(p, 8))**

1) Creamos un puntero auxiliar aux que nos servirá para recorrer la lista. Lo posicionamos al inicio, apuntando al mismo nodo que p.

Nodo* aux=p;

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]

aux



Función: buscar

2) Iniciamos el bucle de búsqueda (Iteración 1). El bucle while se ejecuta mientras se cumplan dos condiciones: que no hayamos llegado al final de la lista (aux != NULL) y que el valor del nodo actual no sea el que buscamos (aux->valor != v). En este punto, aux apunta a 5. Como 5 no es 8, las condiciones se cumplen y entramos al bucle para avanzar al siguiente nodo.

```
while((aux!=NULL) && (aux->valor!=v)){ // Verdadero (aux no es NULL y 5 != 8)
   aux=aux->sig;
}
```

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]



Función: buscar

3) Encontramos el nodo (Fin del bucle). Volvemos a evaluar la condición del while. aux ahora apunta al nodo 8. La primera parte (aux != NULL) es verdadera, pero la segunda (aux->valor != v, es decir, 8 != 8) es falsa. Como una de las condiciones del && es falsa, el bucle se detiene. ¡Hemos encontrado el nodo!

```
while((aux!=NULL) && (aux->valor!=v)){ // Falso (porque 8 == 8)
```

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]



Función: buscar

4) Terminamos la función, devolviendo el valor actual del puntero aux. En este caso, devuelve la dirección de memoria del nodo que contiene el valor 8.

return aux;

El valor devuelto es un puntero al segundo nodo de la lista.

Función: buscar

Caso B: El valor **NO** es encontrado **(buscar(p, 4))**

1), 2) y 3): Recorremos toda la lista. El proceso comienza igual, pero esta vez el bucle while **no** se detendrá. Después del último avance, aux tomará el valor del sig del nodo 7, qué es NULL.

El bucle while se ejecutará 3 veces.

p -> [5 | sig] -> [8 | sig] -> [7 | NULL]

aux (ahora es NULL)

Función: buscar

4) Volvemos a evaluar la condición del while. La primera parte (aux != NULL) ahora es **falsa**, porque aux es NULL. Esto detiene el bucle inmediatamente.

```
while((aux!=NULL) && (aux->valor!=v)){ // Falso (porque aux es NULL)
```

5) Terminamos la función, devolviendo el valor actual del puntero aux. Cómo hemos llegado al final de la lista sin encontrar el valor 4, aux es NULL. La función devuelve NULL, indicando que la búsqueda no tuvo éxito.

```
return aux; //El valor devuelto es NULL.
```

```
// Prototipado
Nodo* buscar(Nodo* p,int v){ //Retorna un puntero al nodo si lo encuentra, o NULL si no.

//Desarrollo de la función
Nodo* buscar(Nodo* p,int v){
        // creamos un auxiliar para recorrer la lista
        Nodo* aux=p;
        // buscamos el nodo que contenga el valor v
        while((aux!=NULL) && (aux->valor!=v)){
                // avanzamos al siguiente nodo
                aux=aux->sig;
        }
        // si lo encontramos devuelve el nodo, sino devuelve NULL
        return aux;
}
```

Función: insertar ordenado

Esta función permite agregar un nuevo nodo a la lista, pero no al final, sino en la posición que le corresponde para que la lista mantenga un orden ascendente.

Estado Inicial de la Lista (debe estar ordenada para que la función tenga sentido):

p -> [10 | sig] -> [20 | sig] -> [40 | NULL]

Hay 3 casos:

- Caso A: insertar en el medio
- Caso B: insertar al principio
- Caso C: insertar al final

Función: insertar ordenado - Caso A

Caso A: Insertar en el medio **(insertarOrdenado(&p, 30))**

1) Creamos el nodo nuevo y los punteros auxiliares, aux para el nodo actual y ant para el nodo anterior, que irá un paso por detrás.

```
Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
nuevo->valor = 30;
nuevo->sig = NULL;

Nodo* aux = *p;
Nodo* ant = NULL;
```

Función: insertar ordenado - Caso A

ant -> NULL

p -> [10 | sig] -> [20 | sig] -> [40 | NULL]

aux

[30 | NULL]

nuevo

Función: insertar ordenado - Caso A

2) Buscamos la posición correcta con el bucle while. Este avanzará mientras el nodo aux exista y su valor sea menor o igual al valor que queremos insertar.

- Iteración 1: aux está en 10. 10 <= 30 es verdadero. Avanzamos: ant se mueve a 10, aux a 20.
- Iteración 2: aux está en 20. 20 <= 30 es verdadero. Avanzamos: ant se mueve a 20, aux a 40.
- Iteración 3: aux está en 40. 40 <= 30 es falso. La condición se rompe y el bucle termina.

Función: insertar ordenado - Caso A

```
while((aux != NULL) && (aux->valor <= v)){ // v es 30
    ant = aux;
    aux = aux->sig;
}
```

p -> [10 | sig] -> [20 | sig] -> [40 | NULL]



Función: insertar ordenado - Caso A

3) Encontramos la posición donde debemos conectar el nuevo nodo. Primero, verificamos si ant es NULL. No lo es (apunta a 20), así que entramos en el else. Hacemos que el sig del nodo anterior (ant) apunte a nuestro nuevo nodo.

```
if(ant == NULL){
    // No se ejecuta
}else{
    ant->sig = nuevo;
}
```



Función: insertar ordenado - Caso A

4) Realizamos la conexión con el siguiente. Hacemos que el sig de nuestro nuevo nodo apunte a aux (el nodo donde nos detuvimos). Esto completa la inserción.

nuevo->sig = aux;

p -> [10 | sig] -> [20 | sig] -> [30 | sig] -> [40 | NULL]

Función: insertar ordenado - Caso B

Caso B: Insertamos al principio **(insertarOrdenado(&p, 5))**

1) Creamos el nodo y los punteros auxiliares. En este caso creamos el nodo 5.

ant -> NULL

p -> [10 | sig] -> [20 | sig] -> [40 | NULL]



aux

[5 | NULL]



nuevo

Función: insertar ordenado - Caso B

2) Buscamos la posición correcta. El bucle while evalúa la condición. aux está en 10. 10 <= 5 es falso. La condición falla en la primera comprobación. El bucle no se ejecuta ni una sola vez.

while((aux != NULL) && (aux->valor <= v)){ ... } // Falso de inmediato.

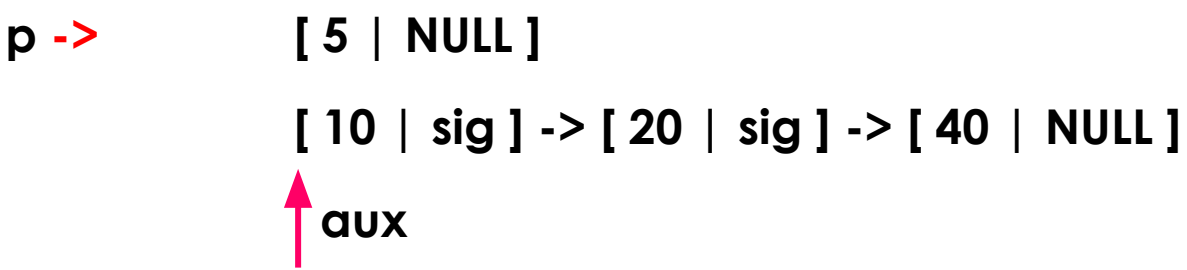
ant sigue siendo NULL

aux sigue apuntando al nodo 10.

Función: insertar ordenado - Caso B

3) Verificamos que ant es NULL. Esto nos indica que debemos insertar al principio. Entramos en el if y hacemos que el puntero principal de la lista, p, apunte directamente a nuestro nuevo nodo.

```
if(ant == NULL){
    *p = nuevo;
}else{ ... }
```



Función: insertar ordenado - Caso B

4) Hacemos que el sig de nuestro nuevo nodo apunte a aux, que era el antiguo primer nodo.

nuevo->sig = aux;

p -> [5 | sig] -> [10 | sig] -> [20 | sig] -> [40 | NULL]

Función: insertar ordenado - Caso C

Caso C: Insertar al final **(insertarOrdenado(&p, 50))**

1) Creamos el nodo y los punteros auxiliares
2) Buscamos la posición correcta.

   El bucle while se ejecutará hasta el final de la lista, porque 10 <= 50, 20 <= 50 y 40 <= 50 son todas condiciones verdaderas.

   Después de procesar el nodo 40, ant apuntará a 40 y aux avanzará a 40->sig, que es NULL.

   En la siguiente evaluación del while, la primera condición (aux != NULL) será falsa, y el bucle se detendrá.

Función: insertar ordenado - Caso C

```
while((aux != NULL) && (aux->valor <= v)){ ... }
```

p -> [10 | sig] -> [20 | sig] -> [40 | NULL]



aux -> NULL

Función: insertar ordenado - Caso C

3) y 4) Conectamos los nodos. ant no es NULL (apunta a 40), así que ant->sig = nuevo hace que el nodo 40 apunte al nuevo nodo 50.

nuevo->sig = aux hace que el sig del nuevo nodo 50 apunte a aux, qué es NULL. Esto es perfecto, porque el 50 es ahora el último nodo.

```
// if(ant == NULL) es falso, entra al else
ant->sig = nuevo;
nuevo->sig = aux;
```

p -> [10 | sig] -> [20 | sig] -> [40 | sig] -> [50 | NULL]

Función: insertar ordenado

```
Nodo* insertarOrdenado(Nodo** p, int v){
        // creamos el nuevo nodo con el valor
        Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
        nuevo->valor = v;
        nuevo->sig = NULL;
        // auxiliares para recorrer y guardar el anterior
        Nodo* aux = *p;
        Nodo* ant = NULL;
        // buscamos la posición donde debe insertarse (orden ascendente)
        while((aux != NULL) && (aux->valor <= v)){
                ant = aux;  // guardamos el anterior
                aux = aux->sig; // avanzamos al siguiente
        }
        if(ant == NULL){ // si es el primero, actualizamos el head
                *p = nuevo;
        }else{
                ant->sig = nuevo; // si no, enlazamos el anterior con el nuevo
        }
        nuevo->sig = aux; // enlazamos el nuevo nodo con el siguiente
        return nuevo; // devolvemos el nodo recien insertado
}
```

Función: liberar

Esta función permite eliminar todos los nodos de la lista para prevenir fugas de memoria. Llamada: **liberar(&p);**

1) Comprobamos si la lista está vacía. Como p apunta al nodo 5, no lo está. Entramos al bucle.

```
while( *p!=NULL ){ // Verdadero
```

p -> [5 | sig] -> [7 | NULL]

Función: liberar

2) Guardamos la referencia al siguiente nodo, para ello creamos un puntero temporal **prox** para guardar la dirección del segundo nodo (7), ya que en el siguiente paso vamos a borrar el primer nodo y sin prox perderíamos el resto de la lista.

Nodo* prox=(*p)->sig;

p -> [5 | sig] -> [7 | NULL]



Función: liberar

3) Liberamos (borramos) el nodo actual al que apunta p (el nodo 5), devolviendo su memoria.

free(*p);

El nodo 5 es eliminado, p apunta a memoria inválida temporalmente

(Memoria liberada) [7 | NULL]



prox

Función: liberar

4) Movemos el head de la lista, es decir que hacemos que p apunte al nodo que guardamos en prox.

La lista ahora comienza en el nodo 7.

***p=prox;**

p -> [7 | NULL]

Función: liberar

5) El bucle se repite. Guardamos el sig del nodo 7 (que es NULL) en prox. Liberamos el nodo 7. p se actualiza a prox (que ahora es NULL).

En la siguiente comprobación, *p es NULL y el bucle termina.

```
//prox = (*p)->sig;  --> prox se vuelve NULL
//free(*p);          --> se libera el nodo 7
//*p = prox;         --> p se vuelve NULL
while( *p!=NULL ){ // Falso

p -> NULL
```

```
void liberar(Nodo** p){
    // mientras la lista no está vacía
    while( *p!=NULL ){
        // guardamos la referencia al siguiente nodo
        Nodo* prox=(*p)->sig;
        // liberamos la memoria del nodo actual
        free(*p);
        // avanzamos al siguiente nodo
        *p=prox;
    }
}
```

Limitaciones de la Lista Simple

El recorrido es en una sola dirección.

Preguntas:

- ¿Cómo encontramos el nodo anterior a uno dado?
  - Rta.:Debemos recorrer la lista desde el principio. ¡Ineficiente!
- ¿Cómo recorremos la lista en orden inverso?
  - Rta.: Imposible sin estructuras adicionales.

Solución: **Añadir un segundo puntero en cada nodo.**

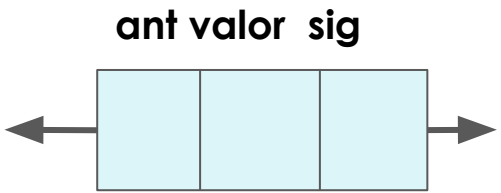
Listas Doblemente Enlazadas

Una lista enlazada donde **cada nodo tiene dos punteros:**

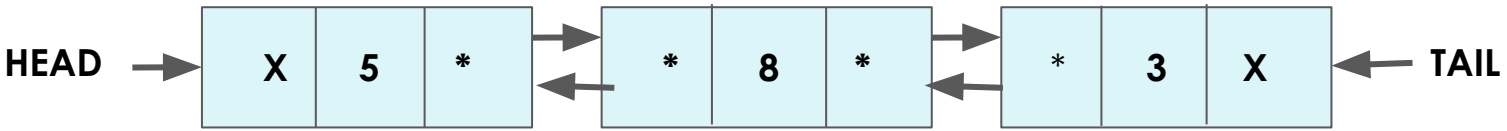
- **ant:** Apunta al nodo anterior.
- **sig:** Apunta al siguiente nodo.

Nueva Estructura del Nodo:

```
typedef struct Nodo {
        int valor;
        struct Nodo* ant; // Anterior
        struct Nodo* sig; // Siguiente
}Nodo;
```



Listas Doblemente Enlazadas



Ahora cada nodo no solo sabe quién es el siguiente, sino también quién fue el anterior. El primer nodo tiene su prev en NULL, y el último tiene su next en NULL.

Función: AgregarAlFinal

Esta función permite agregar un nuevo nodo al final de la lista, asegurando que se establezcan correctamente los dos enlaces: el de ida (sig) y el de vuelta (ant).

Hay 2 casos:

- Caso A: agregamos el primer nodo.
- Caso B: agregamos un nodo a una lista existente.

Función: AgregarAlFinal - Caso A

Caso A: Agregando el primer nodo **(agregarAlFinal(&p, 10))**

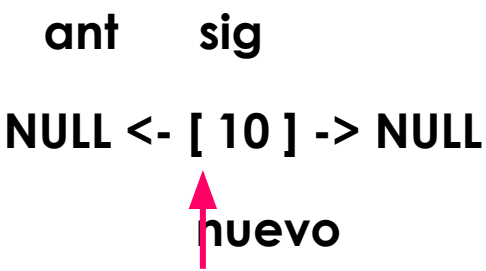
Partimos de una lista completamente vacía.

1) Creamos el nodo y solicitamos memoria. Guardamos el valor 10 en él. Importante: el puntero sig (siguiente) y el puntero ant (anterior) se inicializan a NULL, porque es el único nodo.

Función: AgregarAlFinal - Caso A

```
Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
nuevo->valor = v;  // v es 10
nuevo->sig = NULL;
nuevo->ant = NULL;
```

p -> NULL



Función: AgregarAlFinal - Caso A

2) Comprobamos si la lista está vacía.

if(*p == NULL){ // Verdadero

3) Asignamos el nuevo nodo como el head. Hacemos que p apunte a nuestro nuevo nodo. Este nodo es ahora, simultáneamente, el principio y el fin de la lista.

***p = nuevo;**

ant sig

p -> NULL <- [10] -> NULL

Función: AgregarAlFinal - Caso B

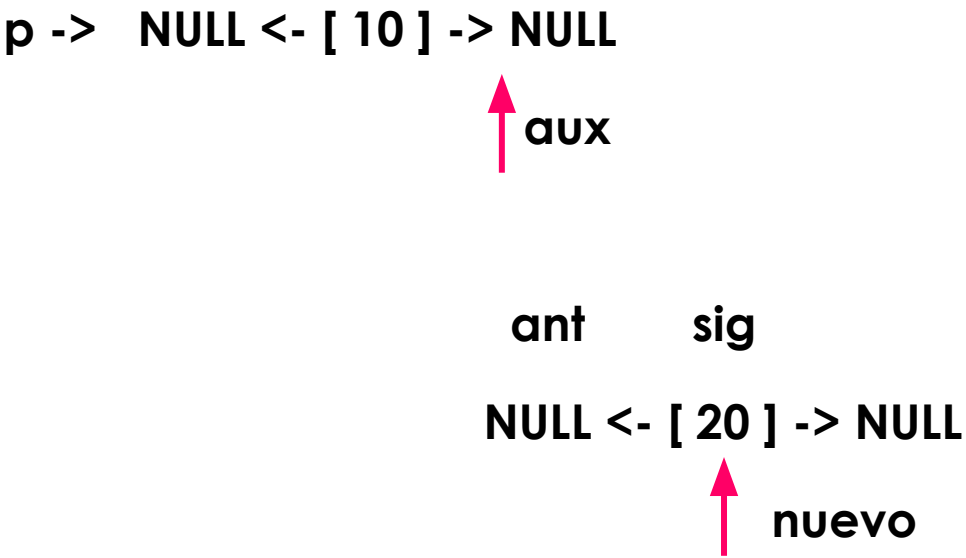
Caso B: Agregando a una lista existente **(agregarAlFinal(&p, 20))**

Partimos de la lista que ya contiene el nodo 10.

1) Creamos nuestro nuevo nodo con el valor 20 (sus punteros ant y sig son NULL por ahora). La condición if(*p == NULL) es falsa, así que entramos en el bloque else. Creamos un **puntero auxiliar** aux para recorrer la lista sin perder la referencia del head.

```
Nodo* nuevo = ...; // Se crea el nodo 20
// if(*p == NULL) es falso, vamos al else
Nodo* aux = *p;
```

Función: AgregarAlFinal - Caso B



Función: AgregarAlFinal - Caso B

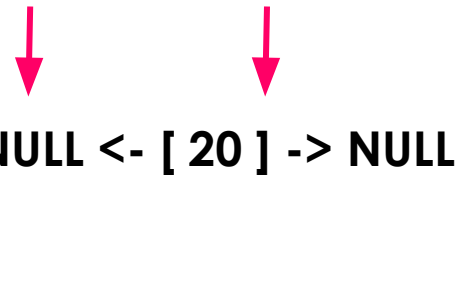
2) Buscamos el último nodo con el bucle while. En este caso, aux está en el nodo 10 y su sig ya es NULL, por lo que la condición del while es falsa de inmediato y el bucle no se ejecuta. aux ya está en la posición correcta.

```
while(aux->sig != NULL){   // Falso
    // No se ejecuta
}
```

Función: AgregarAlFinal - Caso B

3) Establecemos el enlace hacia adelante. Hacemos que el puntero sig del nodo donde está aux (el antiguo último nodo, 10) apunte a nuestro nuevo nodo. Ya podemos ir de 10 a 20.

aux->sig = nuevo;



Función: AgregarAlFinal - Caso B

4) Establecemos el enlace hacia atrás. Hacemos que el puntero ant de nuestro nuevo nodo (20) apunte hacia aux (el nodo 10). Con esto, el enlace bidireccional queda completo. Representado como: **<------>**

nuevo->ant = aux;

p -> NULL <- [10] <------> [20] -> NULL

10->sig apunta a 20

20->ant apunta a 10

```
void agregarAlFinal(Nodo** p, int v){
        // creamos el nuevo nodo y le reservamos memoria
        Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
        // inicializamos los valores del nodo
        nuevo->valor = v;  // guardamos el valor que recibe la función
        nuevo->sig = NULL;  // como sera el ultimo, no apunta a ningun siguiente
        nuevo->ant = NULL; // se actualizara despues si la lista no está vacía

        if(*p == NULL){// si la lista está vacía (head = NULL)
                *p = nuevo; // el nuevo nodo pasa a ser el primero (head)
        }else{
                //Si la lista ya tiene nodos, recorremos hasta el último
                Nodo* aux = *p;  // usamos un auxiliar para no perder el head
                while(aux->sig != NULL){   // avanzamos hasta que aux->sig sea NULL
                        aux = aux->sig;
                }
                // Ahora aux apunta al último nodo actual.
                // Enlazamos el nuevo nodo después del último.
                aux->sig = nuevo;  // el último nodo apunta al nuevo
                nuevo->ant = aux;  // el nuevo nodo apunta hacia atrás al antiguo ultimo
        }
}
```

Función: agregarAlPrincipio

Esta función permite agregar un nuevo nodo al comienzo de la lista, convirtiéndolo en la nueva "cabeza" (head). Esto implica reconfigurar los punteros del antiguo primer nodo para que apunten hacia atrás, al nuevo.

Hay 2 casos:

Caso A: agregamos a una lista está vacía.

Caso B: agregamos a una lista existente.

Función: agregarAlPrincipio - Caso A

Caso A: Agregando a una lista vacía **(agregarAlPrincipio(&p, 5))**

1) Creamos el nuevo nodo con el valor 5. Su puntero ant (anterior) se establece en NULL, lo cual es correcto porque será el primer nodo. Su puntero sig (siguiente) se establece en *p.

   Como la lista está vacía, *p es NULL, así que nuevo->sig también se vuelve NULL. Todo esto es correcto para un nodo que estará solo en la lista.

Función: agregarAlPrincipio - Caso A

```
Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
nuevo->valor = v;   // v es 5
nuevo->sig = *p;  // *p es NULL, asi que nuevo->sig es NULL
nuevo->ant = NULL;
```

p -> NULL

ant sig

NULL <- [5] -> NULL



nuevo

Función: agregarAlPrincipio - Caso A

Paso 2: Comprobamos si la lista no estaba vacía con if(*p != NULL). Como *p es NULL, la condición es falsa, y el bloque de código dentro del if se salta.

Esto es importante para evitar un error, ya que no podemos modificar el puntero ant de un nodo que no existe.

```
if(*p != NULL){ // Falso
    // ... este código se salta
}
```

Función: agregarAlPrincipio - Caso A

Paso 3: Actualizamos el head de la lista. El puntero principal p debe apuntar a nuestro nuevo nodo, estableciéndose como el head de la lista.

***p = nuevo;**

ant sig

p -> NULL <- [5] -> NULL

Función: agregarAlPrincipio - Caso B

Partimos de una lista que ya tiene nodos.

Estado Inicial de la Lista:

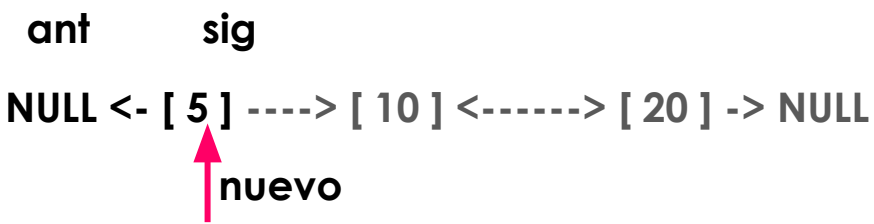
ant sig ant sig

p -> NULL <- [10] <------> [20] -> NULL

Función: agregarAlPrincipio - Caso B

1) Creamos el nuevo nodo con el valor 5. Su puntero ant es NULL (porque va ser le 1er nodo) y el puntero sig se establece en *p, que en este momento apunta al nodo 10. Así, establecemos el primer enlace: el nuevo nodo ya sabe que su siguiente es el antiguo primer nodo.

```
Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));
nuevo->valor = 5;
nuevo->sig = *p;  // nuevo->sig apunta al nodo 10
nuevo->ant = NULL;
```



Función: agregarAlPrincipio - Caso B

2) Actualizamos el puntero ant del antiguo primer nodo. Comprobamos if(*p != NULL). La condición es verdadera, porque p apunta al nodo 10. Entramos en el bloque if.

```
if(*p != NULL){ // Verdadero
   (*p)->ant = nuevo; //se actualiza el puntero ant
}
```

NULL <- [5] <----> [10] <------> [20] -> NULL



Función: agregarAlPrincipio - Caso B

3) Actualizamos el head de la lista. Movemos el puntero principal p para que apunte a nuestro nuevo nodo.

***p = nuevo;**

ant sig ant sig ant sig

p -> NULL <- [5] <------> [10] <------> [20] -> NULL

```
// Agregar al principio de la lista
void agregarAlPrincipio(Nodo** p, int v){

        // creamos el nuevo nodo y le reservamos memoria
        Nodo* nuevo = (Nodo*) malloc(sizeof(Nodo));

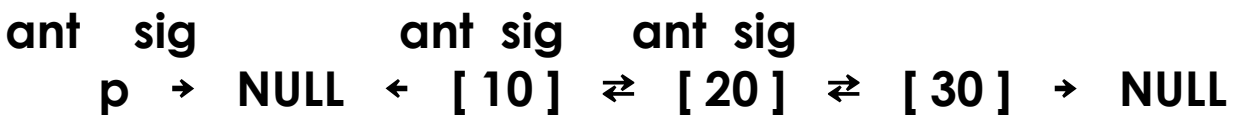
        // inicializamos los valores del nodo
        nuevo->valor = v;   // guardamos el valor recibido por la función
        nuevo->sig = *p;  // el siguiente del nuevo será el nodo actual primero (head)
        nuevo->ant = NULL;  // como será el primer nodo, no tiene nodo anterior

        //si la lista no estaba vacía
        if(*p != NULL){
                // el nodo que antes era el 1ero ahora tiene como anterior al nuevo nodo
                (*p)->ant = nuevo;
        }
        //Actualizamos el puntero al head para que apunte al nuevo nodo
        *p = nuevo;
}
```

Función: eliminar

La función permite encontrar un nodo con un valor específico y removerlo de la lista. En este caso no es necesario crear un puntero "anterior", porque ya lo tenemos.

Estado Inicial:



```
// Eliminar un nodo de la lista doblemente enlazada
// En esta función no necesitamos un nodo auxiliar "anterior" como en la lista simple,
// porque cada nodo ya tiene un puntero al nodo anterior ("ant").
void eliminar(Nodo** p, int v){
        Nodo* aux = *p; //1) Creamos un puntero auxiliar para recorrer la lista
        //2) Buscamos el nodo que contiene el valor v
        // avanzamos mientras no lleguemos al final y no encontremos el valor
        while(aux != NULL && aux->valor != v){
                aux = aux->sig;  // avanzamos al siguiente nodo
        }
        if(aux != NULL){ //3) Si encontramos el nodo a eliminar (aux no es NULL)
                if(aux->ant != NULL){ // 3a) Si el nodo no es el 1ero, actualizamos el enlace del nodo anterior
                        aux->ant->sig = aux->sig; // el anterior apunta al siguiente del nodo eliminado
                }else{
                        *p = aux->sig;  // 3b)  Si el nodo es el primero, movemos el head al siguiente nodo
                }
                // 3c) Si el nodo no es el último, actualizamos el enlace del siguiente nodo
                //  para que apunte al nodo anterior del nodo eliminado
                if(aux->sig != NULL){
                        aux->sig->ant = aux->ant;
                }
                free(aux); // 4) Liberamos la memoria del nodo eliminado
        }
}
```

Lista Simple vs Doble

Características	Lista Simple	Lista Doble
Memoria	Usa menos memoria (un puntero por nodo)	Usa más memoria (dos punteros por nodo)
Recorrido	Solo hacia adelante	Hacia adelante y hacia atrás
Inserción/Eliminación	Más complejo (requiere puntero anterior)	Más simple (si se tiene el nodo a eliminar)
Complejidad (Tiempo)	Puede tardar más porque siempre se empieza desde el inicio	En algunos casos puede ser más rápida porque se puede retroceder

¿Cuándo usar cada una?

Se usa una **Lista Simplemente Enlazada** cuando:

- La memoria es una restricción crítica.
- Solo se necesita iterar sobre la lista en una dirección.
- se implementa estructuras como una Pila (LIFO), donde todas las operaciones ocurren en un extremo (el head).

Se usa una **Lista Doblemente Enlazada** cuando:

- Se necesita recorrer la lista en ambas direcciones.
- Se realizan inserciones o eliminaciones frecuentes en medio de la lista.
- Se Implementa una Cola (Queue) con inserciones por un lado y borrados por el otro de forma eficiente.

📚 Bibliografía sugerida

Sznajdleder, N. (2012). Algoritmos a fondo. Buenos Aires: Alfaomega.

Cap. 11 Estructuras de datos dinámicas lineales